Code review — andrei-backtracking

Runde de review pe proiectul tău, cea mai nouă sus.

Runda 2 — 2026-10-23 — sha ef9e60e

Ce am verificat: cele trei commit-uri de după runda 1 (`f5c8ec7` „permutari-8", `efa9d18` „corectare code review", `ef9e60e` „final tema combinari") și fișa nouă, `exercitii-combinari.pdf`. Fiecare fișier `.h` l-am compilat separat și l-am rulat nu doar pe exemplul din fișă, ci pe toate valorile n = 1…6 (la combinări și toate p), comparând cu lista corectă.

Problema	Fișierul tău	Rezultat
Permutări 1, 3, 4, 6	`permutari_1/3/4/6.h`	✅ (la 1, vezi M1 pentru n=1)
Permutări 2	`permutari_2.h`	✅ B3 reparat: `2 3 1` / `3 2 1`
Permutări 5	`permutari_5.h`	🔴 tot nu compilează (B6)
Permutări 7	`permutari_7.h`	🔴 nu compilează, `if` neterminat (B6)
Permutări 8	`permutari_8.h`	✅ toate n = 1…6
Combinări 1, 2, 3, 4	`combinari_1…4.h`	✅ toate n și p
Combinări 5, 6	`combinari_5/6.h`	🟡 rezultate corecte, dar lipite: `12` în loc de `1 2` (M4)
Combinări 7	`combinari_7.h`	🔴 copia șablonului: 20 de rânduri în loc de 3 (B8)
Combinări 8	`combinari_8.h`	🔴 copia șablonului: nu afișează nimic (B8)

La combinări ai prins ideea din prima: condiția `x[k] > x[k-1]` face ordinea crescătoare, iar fiecare problemă adaugă doar o regulă în plus. 1, 2 și 4 sunt scrise exact așa. Permutările 8 e corectă pentru toate n, iar acolo ai dat funcțiilor nume proprii (`valid8` , `back8`), cum cerea runda 1.

🔴 Critice

B6 🔴 Două fișiere de permutări tot nu compilează — `permutari_5.h:50` , `permutari_7.h:17`

CUM E ACUM:

```
permutari_5.h:50: error: use of undeclared identifier 'back'; did you mean 'back5'?
permutari_7.h:17: error: expected expression        if(x[k]==1&&)
```

`main.cpp` include doar `combinari_6.h` , deci Build-ul din Code::Blocks trece, iar aceste două erori nu le vezi.

DE CE: e B1 + B4 din runda 1, rămase. Compilatorul vede doar ce ajunge prin `#include` în `main.cpp` . Un `.h` din proiect, dar neinclus, nu e compilat niciodată, oricât de greșit ar fi. Commit-ul „corectare code review" a redenumit fișierele, dar `back(1)` de la 5 e neatins, iar la 7 condiția a rămas la jumătate (`x[k]==1&&` fără a doua parte). Un fișier e „gata" abia după ce l-ai inclus, l-ai compilat și i-ai văzut ieșirea.

FIX: la 5, `back5(1);` . La 7, ideea din runda 1: când pui `n` pe poziția `k` , 1 trebuie să fie deja printre `x[1..k-1]` .

B7 🔴 Redenumirea fișierului nu repară garda — `permutari_1.h:1-2` (+ `permutari_2.h` … `permutari_8.h` , liniile 1-2)

CUM E ACUM: am pus două fișiere în același program:

```
#include "permutari_1.h"
#include "permutari_3.h"
int main(){ permutari3(); }
```

```
error: use of undeclared identifier 'permutari3'
```

Exact eroarea din runda 1.

DE CE: garda nu e numele fișierului, e numele macro-ului scris **înăuntru**, pe liniile 1-2. Ai schimbat `permutari-1.h` în `permutari_1.h` , dar înăuntru a rămas `#ifndef PERMUTARI-1_H_INCLUDED` . Preprocesorul nu știe cum se cheamă fișierul. Citește doar textul, se oprește la minus și definește tot macro-ul `PERMUTARI` , comun tuturor celor 8. La combinări ai scris garda corect (`COMBINARI_1_H_INCLUDED`), dar acolo apare a doua jumătate a lui B2: fiecare fișier declară din nou `n` , `x` , `p` și `validCombinari` . `combinari_1.h` + `combinari_2.h` în același program dă `redefinition of 'n'` .

FIX: `_` în loc de `-` pe liniile 1-2 din toate cele 8 permutări. Partea comună (`n` , `x` , `p` , afișarea) într-un singur `.h` , inclus de toate. Fiecare problemă cu funcțiile ei, cu nume proprii, ca la `permutari_8.h` . Abia atunci `main.cpp` le poate include pe toate și le poate rula pe rând.

B8 🔴 Combinările 7 și 8 sunt copia șablonului — `combinari_7.h:11-46` , `combinari_8.h:10-45`

CUM E ACUM:

```
7:  6 3 10  ->  1 2 3 / 1 2 4 / 1 2 5 … 20 de rânduri (fișa: 3)
8:  3       ->  nimic (fișa: 7 rânduri, de la 1 la 3)
```

DE CE: e același lucru ca la permutări 7 în runda 1. Un fișier copiat din șablon compilează și afișează ceva, așa că pare rezolvat. La 7, `S` nu e nici citit, nici verificat. La 8, `combinari()` citește și `p` , dar fișa dă doar `n` : programul așteaptă un al doilea număr, iar la sfârșitul intrării `p` rămâne 0, deci `k==p` nu e adevărat niciodată și nu se afișează nimic. Commit-ul se cheamă „final tema combinari". Singura verificare care prinde asta e cea din runda 1: ieșirea ta lângă cea din fișă, rând cu rând.

FIX: la 7, citești și `S` și afișezi doar când suma celor `p` elemente e `S` . La 8, nu mai ai un `p` dat: fiecare `x[1..k]` crescător e deja o submulțime, deci o afișezi la fiecare pas, nu doar la `k==p` .

🟡 Importante

- **M4** `combinari_5.h:30` , `combinari_6.h:28` : `cout<<x[i]` fără spațiu. Rezultatele sunt corecte, dar ies `12` , `14` , nu `1 2` , `1 4` ca în fișă. La `n` ≥ 10 nu s-ar mai putea citi deloc.
- **M1** (din runda 1, rămas) `permutari_1.h` , `permutari_2.h` : pentru n=1 nu afișează nimic (trebuia `1`).
- **M2** (din runda 1, rămas) Numele funcțiilor: `permutari()` la 1 și 7, `combinari()` în toate cele 8. Doar `permutari_8.h` urmează regula.
- **M3** (din runda 1, rămas) `abs` fără `#include <cstdlib>` : `permutari_5.h:16` , `permutari_8.h:13` .

🟢 Cleanups

- **C5** `combinari_7.h:4-6` : `#include <iostream>` și `using namespace std;` de două ori.
- **C6** `permutari_8.h:7` : `int valid8` întoarce `true` / `false` , deci `bool` . Comentariul rămas la liniile 9-12 se șterge.
- **C4** (rămas) Commit-urile tot ca `KERNELOS-PC\Administrator` .

Before / After

	Acum	Corect
B6	`back(1);` (`permutari_5.h:50`)	`back5(1);`
B6	`if(x[k]==1&&)`	condiția „1 e deja printre `x[1..k-1]`" când `x[k]==n`
B7	`#ifndef PERMUTARI-1_H_INCLUDED`	`#ifndef PERMUTARI_1_H_INCLUDED`
B7	`int n, x[20]; int p;` în fiecare fișier	o singură dată, într-un `.h` comun
B8	`combinari_7.h` = șablonul	citește `S`, afișează doar suma egală cu `S`
B8	`combinari_8.h` = șablonul, citește `p`	citește doar `n`, afișează la fiecare `k`

Ce urmează

1. B6: `back5(1)` și condiția de la 7, apoi rulezi 5 și 7 cu datele din fișă.
2. B8: combinări 7 și 8, rulate și comparate rând cu rând.
3. B7: garda cu `_` și partea comună într-un singur `.h`. Abia apoi un `main.cpp` care le include pe toate.

Întrebări

1. Ai redenumit `permutari-1.h` în `permutari_1.h`. De ce eroarea cu `permutari3` a rămas aceeași?
2. La combinări 8, de ce nu trebuie să mai aștepți `k==p` ca să afișezi?
3. De ce a trecut Build-ul în Code::Blocks, deși `permutari_5.h` are o eroare?

Runda 1 — 2026-10-15 — sha a8e1fc0

- 🔴 **B1** un singur `.h` compilat: neatins (B6)
- 🔴 **B2** garda cu minus + `n` / `x` declarate în fiecare fișier: fișierele redenumite, garda neatinsă (B7)
- ✅ **B3** problema 2 pune 1 de două ori: închis (rulat n = 2…6)
- 🔴 **B4** `back(1)` la 5: neatins (B6)
- ◐ **B5** problema 8 ✅ scrisă și corectă · problema 7 🔴 nu compilează (B6)
- ❌ M1, M2, M3, C2 (`permutari.h`), C4 neatinse · C1 urmărirea de mână ✅